

情報通信実験2講義資料 情報理論後半

4. 記号列に対する符号化と情報源

記号列に対する符号化

- ▶ これまでは記号毎の符号化を考えてきたが、記号列毎の符号化を考えることができる
- ▶ 考え方は簡単で、記号列を一つの記号とみて符号化すればよい

表: 符号語の例 (情報源アルファベット: $\{a, b\}$)

記号列	符号語	符号語長
<i>aa</i>	0	1
<i>ab</i>	10	2
<i>ba</i>	110	3
<i>bb</i>	111	3

記号列に対する平均符号語長とエントロピー

- ▶ 情報源アルファベット： $\{x_1, x_2, \dots, x_m\}$
- ▶ 長さ n の記号列： $\mathbf{x} = (x_{i_1}, x_{i_2}, \dots, x_{i_n})$
- ▶ 記号列 $\mathbf{x}$ の生起確率： $P_n(\mathbf{x})$
- ▶ $\mathbf{x}$ の符号語長： $l(\mathbf{x})$

$$\text{平均符号語長： } L_n \triangleq \sum_{\mathbf{x}} P_n(\mathbf{x}) l(\mathbf{x})$$

$$\text{エントロピー： } H(P_n) \triangleq - \sum_{\mathbf{x}} P_n(\mathbf{x}) \log_2 P_n(\mathbf{x})$$

平均符号語長の理論限界

どんな語頭符号も

$$H(P_n) \leq L_n$$

を満たす。

情報源と記号の生起確率

- ▶ 情報源：記号列の生起確率 P_n を特徴付ける数学的なモデル
- ▶ 無記憶情報源：記号列の中の任意の位置の記号がそれ以前や以後の記号とは独立に、ある生起確率に従ってランダムに現れる

$$P_n(\mathbf{x}) = P(x_{i_1})P(x_{i_2}) \cdots P(x_{i_n})$$

ここで、 $P(x)$ は 1 記号 x が生起する確率

- ▶ マルコフ情報源：記号列の中の任意の位置の記号がそれ以前の記号に依存して、ある生起確率に従ってランダムに現れる

$$P_n(\mathbf{x}) = P(x_{i_1})P(x_{i_2}|x_{i_1}) \cdots P(x_{i_n}|x_{i_{n-1}})$$

ここで、 $P(x)$ は 1 記号 x が生起する確率であり、 $P(y|x)$ は x が生起した後に 1 記号 y が生起する確率[†]

[†]条件付き確率

無記憶情報源の例

表: 1 記号の確率

記号	$P(x)$
a	0.2
b	0.8

$n = 2$ の場合 :

$$P_2(aa) = P(a)P(a) = 0.04$$

$$P_2(ab) = P(a)P(b) = 0.16$$

$$P_2(ba) = P(b)P(a) = 0.16$$

$$P_2(bb) = P(b)P(b) = 0.64$$

マルコフ情報源の例

表: 1 記号の確率

記号	$P(x)$
a	0.2
b	0.8

記号 x	記号 y	$P(y x)$
a	a	0.2
a	b	0.8
b	a	0
b	b	1

$n = 2$ の場合 :

$$P_2(aa) = P(a)P(a|a) = 0.04$$

$$P_2(ab) = P(a)P(b|a) = 0.16$$

$$P_2(ba) = P(b)P(a|b) = 0$$

$$P_2(bb) = P(b)P(b|b) = 0.8$$

5. 算術符号

ハフマン符号の問題点

一度に符号化する記号列の長さが十分長ければ、1記号あたりの平均符号語長が改善する：

$$\frac{H(P_n)}{n} \leq \frac{\text{平均符号語長}}{n} \leq \frac{H(P_n)}{n} + \frac{1}{n}$$

ところが、記号列が長いとハフマン符号の木が大きくなりすぎて実行するのが難しい[†]。

[†]50文字をまとめて符号化するには、たとえ記号数2だとしても 2^{50} の葉が必要になる

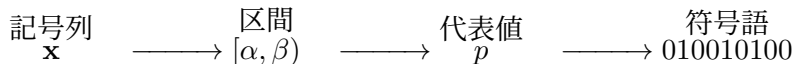
算術符号

- ▶ 記号列長に比例する程度の計算量で、記号列全体を一つの符号語にする実用的な符号化法
- ▶ 実数 0 と 1 の間の実数の区間によって記号列を表すというアイデアを用いる
- ▶ 符号化の際には、この区間の代表値を表すビット列を符号語として出力する
- ▶ 復号化の際には、符号語から得られる代表値について、その値を含む区間に対応する記号列を出力する

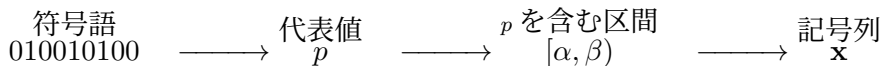
以下では無記憶情報源に対する算術符号を紹介する。

符号化と復号化の概要

符号化の概要：



復号化の概要：



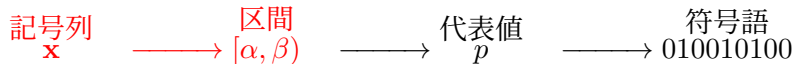
1 記号に対する区間

区間 $[0, 1)$ を各記号の生起確率に比例した区間に分割し、それらの区間がそれぞれの記号を表していると考ええる。

表: 各記号の生起確率と対応する区間 (情報元アルファベット: $\{a, b, c, _ \}$)

記号	生起確率	区間	区間幅
a	0.2	$[0, 0.2)$	0.2
b	0.2	$[0.2, 0.4)$	0.2
c	0.4	$[0.4, 0.8)$	0.4
$_$	0.2	$[0.8, 1)$	0.2

記号列に対する区間

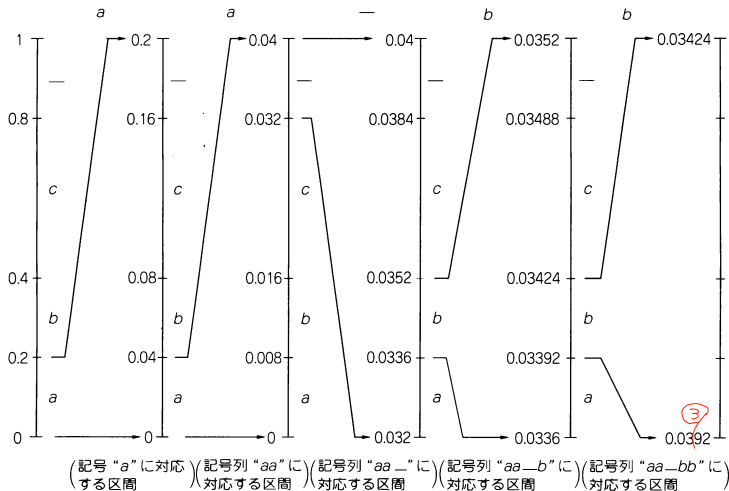


以下の手順で最終的に採用される区間を記号列に対する区間とする：

- ① 初期状態として区間 $[0, 1)$ が採用されているとする
- ② 記号列から 1 記号だけ読み込む。読み込む記号がなければ現時点で採用されている区間を出力する
- ③ 採用されている区間を各記号の生起確率に比例する区間に分割する
- ④ 2 で読み込んだ記号に対する区間を、3 で分割した区間の中から採用する
- ⑤ 2、3、4 を繰り返す

記号列に対する区間の例 (つづく)

記号列 aa_bb_cccc に対する区間[†]:



[†]図は [植松友彦, 文章データ圧縮アルゴリズム入門, CQ 出版, 1994] からの引用

記号列に対する区間の例

記号列 aa_bb_cccc に対する区間：

入力記号	区間の下限	区間の上限	区間幅
a	0	0.2	0.2
a	0	0.04	0.04
—	0.032	0.04	0.008
b	0.0336	0.0352	0.0016
b	0.03392	0.03424	0.00032
—	0.034176	0.03424	0.000064
c	0.0342016	0.0342272	0.0000256
c	0.03421184	0.03422208	0.00001024
c	0.034215936	0.034220032	0.000004096
c	0.0342175744	0.0342192128	0.0000016384

代表値の選択と符号語



▶ $\mathbf{x}$ に対応する区間 $[\alpha, \beta)$ の中点: $m(\mathbf{x}) = \frac{1}{2}(\alpha + \beta)$

▶ $l(\mathbf{x}) = \lceil -\log P_n(\mathbf{x}) \rceil + 1$
($P_n(\mathbf{x}) =$ 区間 $[\alpha, \beta)$ の区間幅)

$m(\mathbf{x})$ に対応する二進数表現の小数点以下 $l(\mathbf{x})$ 桁を符号語とする[†]:

$$m(\mathbf{x}) = 0.b_1b_2 \cdots b_{l(\mathbf{x})} \cdots$$

のとき、

$$\mathbf{x} \text{ の符号語} = b_1b_2 \cdots b_{l(\mathbf{x})}$$

[†]したがって、代表値は $0.b_1b_2 \cdots b_{l(\mathbf{x})}$

代表値の選択と符号語の例

記号列 aa_bb_cccc に対する区間：

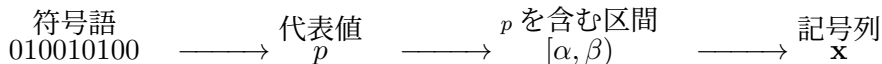
区間の下限	区間の上限	区間幅
0.0342175744	0.0342192128	0.0000016384

記号列 aa_bb_cccc に対する符号語：

$$\begin{aligned}m(\mathbf{x}) &= \frac{1}{2}(0.0342175744 + 0.0342192128) \\&= 0.0000100011000010100010 \dots \quad (\text{二進数}) \\l(\mathbf{x}) &= \lceil -\log_2 0.0000016384 \rceil + 1 = 21\end{aligned}$$

なので、符号語は $\underbrace{000010001100001010001}_{21}$

算術符号の復号化



記号列の長さを知っていることを仮定する。

- ① 符号語 $b_1b_2 \cdots b_l$ に対応する代表値 $p = 0.b_1b_2 \cdots b_l$ を得る
- ② p を含む区間に対応する 1 記号 x を出力する
- ③ p を更新する：

$$p \leftarrow \frac{p - x \text{ に対応する区間の下限}}{x \text{ に対応する区間幅}}$$

- ④ 記号列の長さだけ記号を出力するまで 2 と 3 を繰り返す

算術符号の復号化の例

表: 各記号の生起確率と対応する区間

記号	生起確率	区間	区間幅
a	0.2	$[0, 0.2)$	0.2
b	0.2	$[0.2, 0.4)$	0.2
c	0.4	$[0.4, 0.8)$	0.4
—	0.2	$[0.8, 1)$	0.2

- ① 符号語 000010001100001010001 に対応する代表値は

$$p = 0.000010001100001010001_2 = 0.034218311309814453125$$

- ② $p = 0.034218311309814\dots$ を含む区間に対応する記号 a を出力する

- ③ p を更新する:

$$p \leftarrow \frac{p-a \text{ に対応する区間の下限}}{a \text{ に対応する区間幅}} = 0.171091556549072265625$$

- ④ $p = 0.171091556549072\dots$ を含む区間に対応する記号 a を出力する

- ⑤ p を更新する:

$$p \leftarrow \frac{p-a \text{ に対応する区間の下限}}{a \text{ に対応する区間幅}} = 0.855457782745361328125$$

⋮

Jonese 符号 (課題で扱う予定)

算術符号にはいくつかの実用上の問題点がある：

- ▶ 算術符号の区間の計算は入力する記号列が長くなるにつれて、より多くの桁数が必要になる
- ▶ 浮動小数点演算を行う際の丸め誤差 (有効数字への四捨五入) の影響がある

課題で扱う予定の Jones 符号ではこれらの問題点を解決して、有限精度の計算で算術符号を実装している。

6. LZ78 符号

記号の確率分布が与えられていない場合

- ▶ これまでは記号あるいは記号列の確率分布が与えられていることを前提として符号化を行ってきた
- ▶ 記号列の確率分布が与えられていなくても、記号列長が長くなるにつれて、1記号あたりの平均符号語長がエントロピーに収束するような符号は構成できるのか？

LZ78 符号

- ▶ 記号を読み込むたびに、木で表される「辞書」を動的に作成し、その辞書を参照しながら符号化を行う
- ▶ 記号列の確率分布が与えられていなくても、記号列の長さが長くなるにつれて、1 記号あたりの平均符号語長がエントロピーに近づいていく †
- ▶ Ziv と Lempel によって 1978 年に発表されたことにちなんで LZ78 符号と呼ばれている

†このような符号はユニバーサル符号と呼ばれることがある。

‡どんな情報源についてもこれが成り立つわけではなく、無記憶情報源といった特定の情報源についてのみ成り立つことに注意。

LZ78 符号における符号化と復号化の概要

▶ 符号化：

- ▶ 符号化する記号列に最長一致する記号列を辞書の中から探し、その記号列の「辞書での参照番号」と辞書には登録されていなかった、最長一致した記号列の「次の 1 記号」を符号語とする
- ▶ また同時に、最長一致した記号列に「次の 1 記号」を加えた記号列を辞書に登録することで、動的に辞書を更新していく

▶ 復号化：

- ▶ 符号語から得られる、「辞書での参照番号」と「次の 1 記号」から、辞書を参照して記号列を復号する
- ▶ また同時に、「辞書での参照番号」の記号列に「次の 1 記号」を加えた記号列を辞書に登録することで、動的に辞書を更新していく

以下では記号列 `ababbababaa` に対する符号化と復号化の例を用いて、LZ78 符号を説明する。ただし、図は [植松友彦, 文章データ圧縮アルゴリズム入門, CQ 出版, 1994] からの引用である。

LZ78 符号の符号化 (つづく)

- ① 辞書を表す木は根だけであるとする
- ② 1 番目の部分列の符号化: 最初の記号 “a” から始まる記号列と最長一致する記号列を辞書から探す。このとき、“0” (根の節点番号。辞書に記号が無いことを表している) と最初の 1 記号 “a” を並べた (0, “a”) を「中間符号語」とし、(0, “a”) に対する適当なビット列を符号語とする。ただし、最初は中間符号語の第一要素が必ず “0” になるので、記号 “a” に対する適当なビット列 (例えば ASCII コード) のみを符号語とする

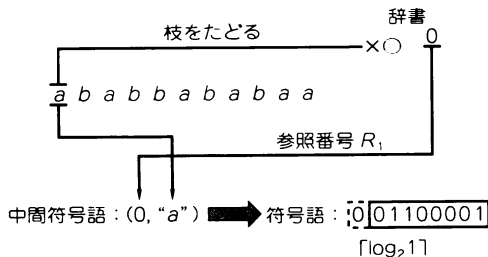


図: 記号 “a” の符号化

LZ78 符号の符号化 (つづく)

- ③ 辞書を表す木に新たな節点を作り、この節点に番号“1”を与える。
また、節点1と根を枝で結び、この枝にラベル“a”をつけることで、
辞書に記号“a”を登録する

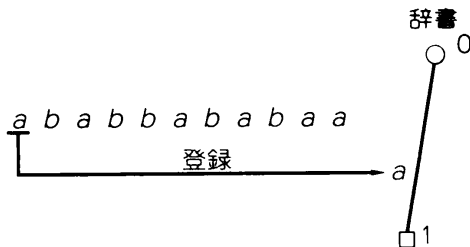


図: 記号“a”の辞書への登録

LZ78 符号の符号化 (つづく)

- ④ 2 番目の部分列の符号化: 入力記号列の 2 番目の記号から始まる記号列 babbababaa に従って、根から枝のラベルをたどる。ラベル “b” をもつ枝はないので、中間符号語として、(0, “b”) を出力する。木の内部節点は 0 か 1 のみであり、中間符号の最初の数値は $\lceil \log_2 2 \rceil$ ビットで表現できるので、(0, “b”) に対する符号語は 001100010 となる

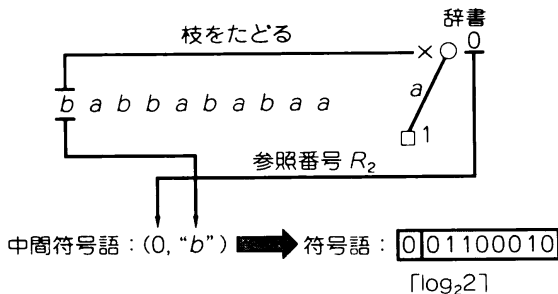


図: 記号 “b” の符号化

LZ78 符号の符号化 (つづく)

- ⑤ 辞書を表す木に新たな節点を作り、この節点に番号 “2” を与える。
また、節点 2 と根を枝で結び、この枝にラベル “b” をつけ、辞書に
新たな記号 “b” を登録する

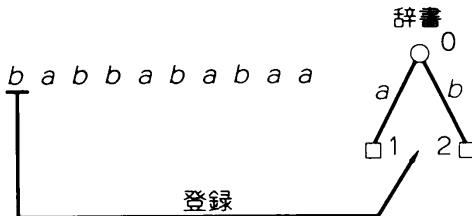
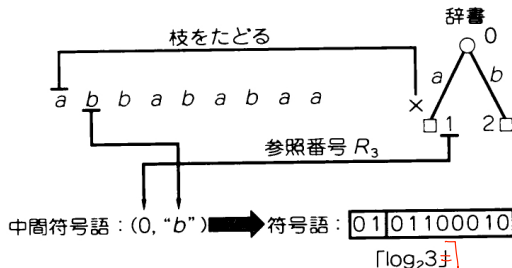


図: 記号 “b” の辞書への登録

LZ78 符号の符号化 (つづく)

- ⑥ 3 番目の部分列の符号化: 入力記号列の 3 番目の記号から始まる記号列 `abbababaa` に従って、根から枝のラベルをたどる。ラベル “a” をたどったところでたどれる枝がなくなるので、中間符号語として、この節点の番号である “1” (“a” の参照番号) と “a” の次の 1 記号 “b” を並べた (1, “b”) を出力する。木の内部節点は 0、1、2 のみであり、中間符号の最初の数値は $\lceil \log_2 3 \rceil$ ビットで表現できるので、(1, “b”) に対する符号語は **01**01100010 となる



LZ78 符号の符号化 (つづく)

- ⑦ 辞書を表す木に新たな節点を作り、この節点に番号“3”を与える。
また、節点1と節点3を枝で結び、この枝にラベル“b”をつけること
で、記号列“ab”を辞書に登録する

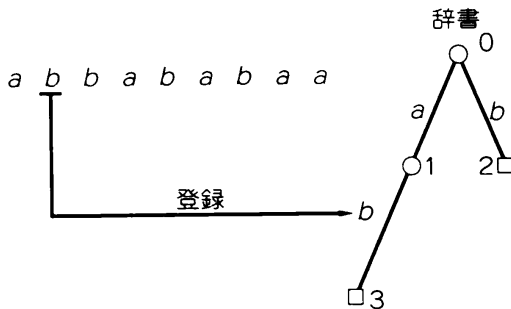


図: 記号“ab”の辞書への登録

LZ78 符号の符号化 (つづく)

- ⑧ 以下、同様な操作を記号列の終わりまで繰り返すと、中間符号語の系列として、(2, "a")、(4, "b")、(1, "a") が出力され、それぞれに対する符号語が得られる

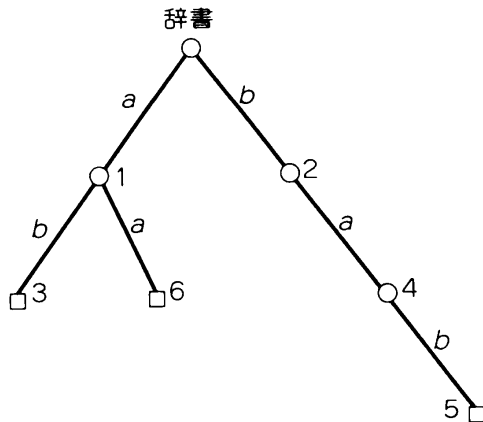


図: 最終的な辞書

LZ78 符号の符号化

例外として、枝をたどっている途中で符号化する記号列の最後まで達し、次の記号がないときは、中間符号語として現時点の節点の参照番号のみを出力し(したがって最終的な符号語は節点番号の2進数表示のみ)、符号化を終了する。

中間符号語のビット列表示 j 番目の部分列の符号化(すなわち、 j 番目の中間符号語の構成時)における辞書の参照番号は必ず $j - 1$ 以下になっています。したがって、中間符号の最初の数字を $\lceil \log_2 j \rceil$ ビットで表現することにします。

abbababaa の符号化結果は、中間符号語の要素の区切りを明らかにして書くと
(0, "a") | (0, "b") | (1, "b") | (2, "a") | (4, "b") | (1, "a")
01100001 | 0,01100010 | 01,01100010 | 10,01100001 | 100,01100010 | 001,01100001
となります。

LZ78 符号の復号化 (つづく)

- ① 辞書を表す木は根だけであるとする
- ② 最初の符号語を復号し、記号 “a” を得る

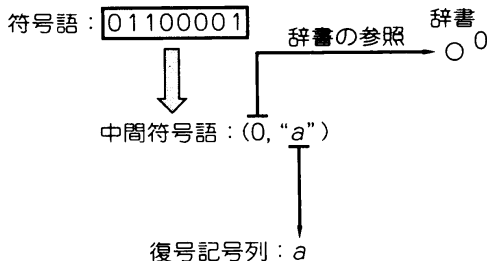


図: 記号 “a” の復号

LZ78 符号の復号化 (つづく)

- ③ 辞書を表す木に新たな節点を作り、番号“1”を与える。また、節点1と根を枝で結び、この枝にラベル“a”をつけ、辞書に記号“a”を登録する



図: 記号“a”の辞書への登録

LZ78 符号の復号化 (つづく)

- ④ 2 番目の符号語を復号して、参照番号 0 と記号 “b” を得る。節点 0 に対応する記号列 (空列) に “b” を続けた “b” を出力する

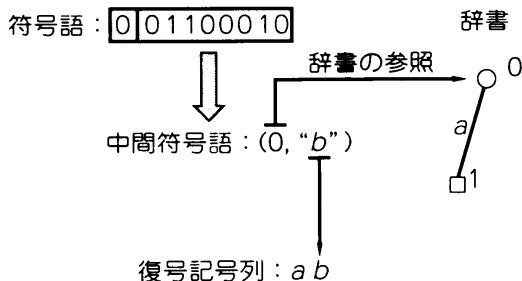


図: 記号 “b” の復号

LZ78 符号の復号化 (つづく)

- ⑤ 辞書を表す木に新たな節点を作り、番号 2 を与える。また、節点 2 と根を枝で結び、この枝にラベル “b” をつけ、辞書に新たな記号 “b” を登録する

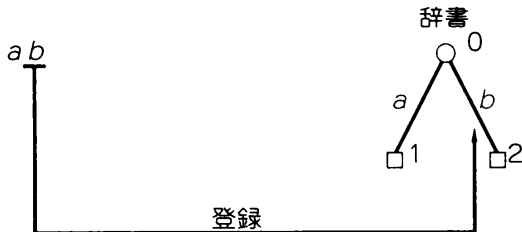


図: 記号 “b” の辞書への登録

LZ78 符号の復号化 (つづく)

- ⑥ 3 番目の符号語を復号して、参照番号 1 と記号 “b” を得る。節点 1 に対応する記号列 (“a”) に記号 “b” を続けた “ab” を符号語に対応する記号列として出力する (j 番目の符号語の参照番号は $\lceil \log_2 j \rceil$ ビットで表されている。)

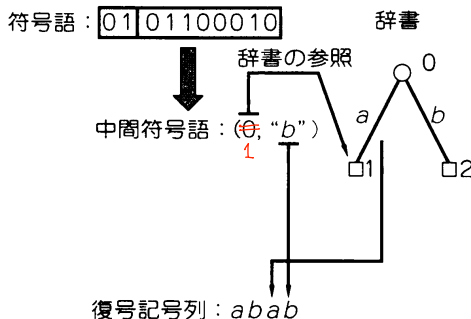


図: 記号 “ab” の復号

LZ78 符号の復号化

- ⑦ 辞書を表す木に新たな節点を作り、番号 3 を与える。また、節点 1 と節点 3 を枝で結び、この枝にラベル “b” をつけ、記号列 “ab” を辞書に登録する
- ⑧ 以下、同様な操作を符号語列の終わりまで繰り返す

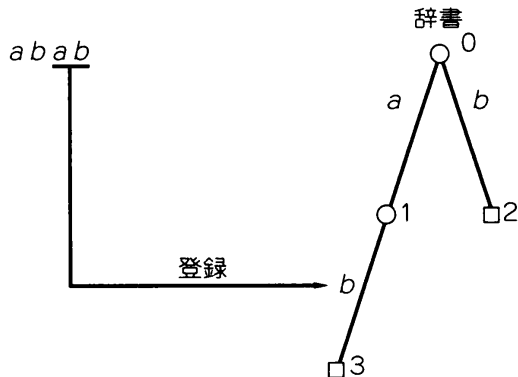


図: 記号 “ab” の辞書への登録